

 Dev ▾

...

gan_project / p8-gan-032023.ipynb

 Daims971 Kaggle Notebook | P8_GAN_032023 | Version 50 

 1 contributor

2.07 MB 

 Open in Kaggle

In [1]:

```
# This Python 3 environment comes with many helpful analytics libraries
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list the files in the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/)
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session
```

Table of content

1. Introduction to GAN
2. Competition context
3. Data preparation
4. Model preparation
 - 4.1. Different type of Neural Networks
 - 4.2. Evaluation of time generation
 - 4.3. Build the GAN
5. Train the GAN
6. Test the GAN
7. Create submission file

1. Introduction to GAN

Nowadays, a new tendency of AI is trending the net: AI that generate images like Midjourney and DALL-E.

Here are some examples of cats generated from the AI:





The main method to do these kind of images is called GAN: Generative Adversarial Network.

Generative Adversarial Networks (GANs) are a type of machine learning algorithm that consists of two neural networks: a generator and a discriminator. The generator creates new data instances while the discriminator evaluates the authenticity of these instances. The two networks are trained simultaneously in a game-like process where the generator learns to create increasingly realistic data instances that can fool the discriminator, and the discriminator learns to become more accurate in distinguishing between real and fake data. Through this iterative process, GANs can produce high-quality, realistic outputs such as images, videos, and even music.

2. Competition context

The main purpose of this competition is to make a GAN that will be able to generate between 7000 and 10000 images with the style of the famous painter Monet.

The generated images have to be jpg format with a size of 256x256x3 (RGB) and be zipped in a single file named images.zip.

The evaluation metric used by kaggle to know if the image is considered good is MiFID: **Memorization-informed Fréchet Inception Distance**.

The smaller MiFID is, the better your generated images are.

More details can be found at: [Kaggle Competition](#).

3. Data Preparation

Based on "Monet CycleGAN Tutorial" notebook from Amy Jang, which is a tutorial starter for this competition, we will prepare the setup and data we need to implement our GAN. It can be found there:

<https://www.kaggle.com/amyjang/monet-cyclegan-tutorial>

This notebook gives basis of how data will be loaded inside a kaggle notebook. The method used in this notebook to generate images is called cycleGAN, that we won't use for the time being.

First of all, we have decided to try executing "Monet CycleGAN Tutorial" notebook to know how it works and how long it takes to train and generate images for the competition.

The notebook takes around 1h to execute. It trains a CycleGAN on 25 epochs with 300 images with each epoch taking around 40s (except for the first one). That notebook generates a bit more than 7000 images that are saved on a images.zip file.

That notebook used a TPU to be run.

An explanation about TPU and how it's used on kaggle can be found here:

[Kaggle TPU documentation](#).

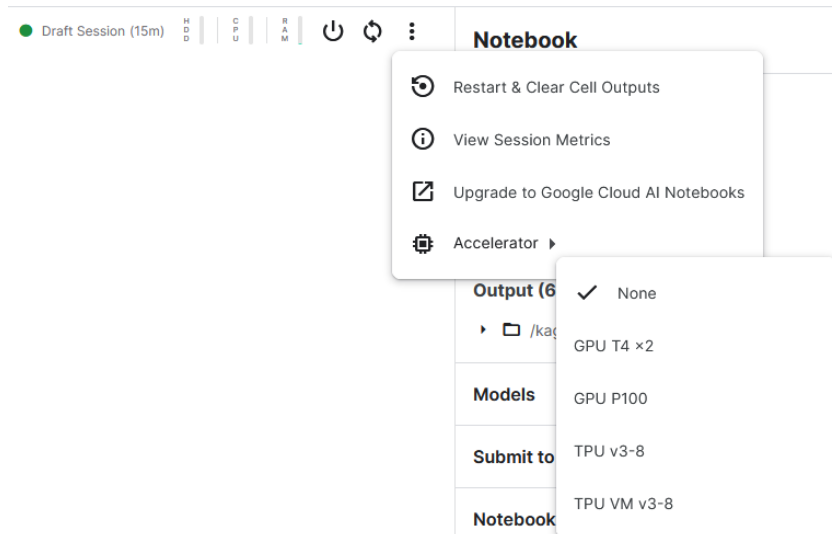
In order to evaluate how TPU is useful for the execution, we tried to run the

notebook without using TPU.

After running without TPU, the same notebook took around 11h to be executed. We could notice that each training epoch took around 1600s compared to 40s with TPU.

Based on that, we choose to use TPU as the same way it's used in Amy Jang notebook.

First, we need to activate the use of TPU (v3-8) on our kaggle notebook like this:



```
In [2]: # Library for Data science

import tensorflow as tf #NN processing
from tensorflow import keras
from tensorflow.keras import layers
import tensorflow_addons as tfa

from kaggle_datasets import KaggleDatasets
import matplotlib.pyplot as plt
import numpy as np # Linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
```

```
In [3]: import glob
import imageio
import matplotlib.pyplot as plt
import os
import PIL
import time

from IPython import display
```

```
In [4]: # Library for TPU acceleration
try:
    tpu = tf.distribute.cluster_resolver.TPUClusterResolver()
    print('Device:', tpu.master())
    tf.config.experimental_connect_to_cluster(tpu)
    tf.tpu.experimental.initialize_tpu_system(tpu)
    strategy = tf.distribute.experimental.TPUStrategy(tpu)
except:
    strategy = tf.distribute.get_strategy()
print('Number of replicas:', strategy.num_replicas_in_sync)

AUTOTUNE = tf.data.experimental.AUTOTUNE

print(tf.__version__)
```

```
Number of replicas: 1
2.6.4
```

Loading Data

The dataset we are using contains four directories: monet_tfrec, photo_tfrec, monet_jpg, and photo_jpg.

The monet_tfrec and monet_jpg directories contain the same painting images sized 256x256 in JPEG format or TFRecord format, and the photo_tfrec and photo_jpg directories contain the same photos also sized 256x256 in JPEG format or TFRecord format.

The monet directories contain Monet paintings that we will use for training our models.

The photo directories contain photos that we will add Monet-style and submit the new generated images.

We want to keep our photo dataset and our Monet dataset separate. First, load in the filenames of the TFRecords.

```
In [5]: GCS_PATH = KaggleDatasets().get_gcs_path() #Get path of data from com
```

```
In [6]: MONET_FILENAMES = tf.io.gfile.glob(str(GCS_PATH + '/monet_tfrec/*.tfre
print('Monet TFRecord Files:', len(MONET_FILENAMES))

PHOTO_FILENAMES = tf.io.gfile.glob(str(GCS_PATH + '/photo_tfrec/*.tfre
print('Photo TFRecord Files:', len(PHOTO_FILENAMES))
```

Monet TFRecord Files: 5
Photo TFRecord Files: 20

All the images for the competition are already sized to 256x256. As these images are RGB images, set the channel to 3. Additionally, we need to scale the images to a [-1, 1] scale. Because we are building a generative model, we don't need the labels or the image id so we'll only return the image from the TFRecord.

```
In [7]: img_sz = 256
IMAGE_SIZE = [img_sz, img_sz]

def decode_image(image):
    image = tf.image.decode_jpeg(image, channels=3)
    image = (tf.cast(image, tf.float32) / 127.5) - 1
    image = tf.reshape(image, [*IMAGE_SIZE, 3])
    return image

def read_tfrecord(example):
    tfrecord_format = {
        "image_name": tf.io.FixedLenFeature([], tf.string),
        "image": tf.io.FixedLenFeature([], tf.string),
        "target": tf.io.FixedLenFeature([], tf.string)
    }
    example = tf.io.parse_single_example(example, tfrecord_format)
    image = decode_image(example['image'])
    return image
```

Define the function to extract the image from the files.

```
In [8]: def load_dataset(filenames, labeled=True, ordered=False):
dataset = tf.data.TFRecordDataset(filenames)
dataset = dataset.map(read_tfrecord, num_parallel_calls=AUTOTUNE)
return dataset
```

Let's load in our datasets.

```
In [9]: batch_size = 32 #128*8
```

```
In [10]: # batch_size: number of consecutive elements of this dataset to combi
# 288 allows to be a multiple of 32

if batch_size ==1 or batch_size ==12 :
    monet_ds = load_dataset(MONET_FILENAMES, labeled=True).batch(batch_size)
    photo_ds = load_dataset(PHOTO_FILENAMES, labeled=True).batch(batch_size)
else:
    monet_ds = load_dataset(MONET_FILENAMES, labeled=True).take(288).
    photo_ds = load_dataset(PHOTO_FILENAMES, labeled=True).take(288).
```

We took one example of each of these lists.

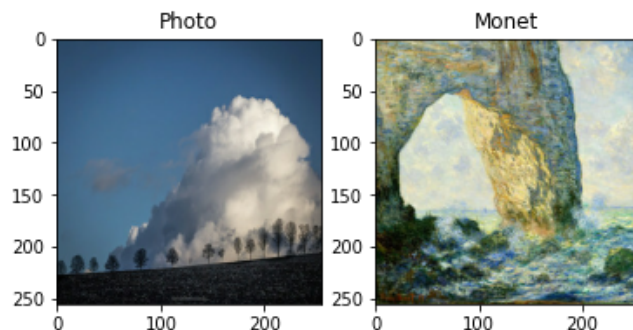
```
In [11]: #next : Get following element of the iter(list) each time it is called
example_monet = next(iter(monet_ds))
example_photo = next(iter(photo_ds))
```

Let's visualize a photo example and a Monet example.

```
In [12]: plt.subplot(121)
plt.title('Photo')
plt.imshow(example_photo[0] * 0.5 + 0.5)

plt.subplot(122)
plt.title('Monet')
plt.imshow(example_monet[0] * 0.5 + 0.5)
```

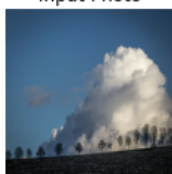
Out[12]: <matplotlib.image.AxesImage at 0x7f44558a1710>



```
In [13]: _, ax = plt.subplots(5, 2, figsize=(12, 12))
for i, img in enumerate(photo_ds.take(5)):
    #print(str(i))
    img = (img[0] * 127.5 + 127.5).numpy().astype(np.uint8)

    ax[i, 0].imshow(img)
    #ax[i, 1].imshow(monet)
    ax[i, 0].set_title("Input Photo")
    #ax[i, 1].set_title("Monet-esque")
    ax[i, 0].axis("off")
    ax[i, 1].axis("off")
plt.show()
```

Input Photo

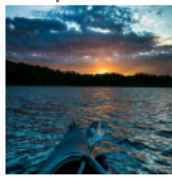


Input Photo





Input Photo



Input Photo



Input Photo



Calcul of number of elements in datasets.

```
In [14]: len_monet_ds = 0
         for i in monet_ds:
             #print(i)
             #print(len(i))
             len_monet_ds = len_monet_ds + len(i)
             #len_monet_ds+=1
```

```
In [15]: len_monet_ds #len_data =
```

Out[15]: 288

```
In [16]: len_photo_ds = 0
         for i in photo_ds:
             #print(i)
             #len_photo_ds+=1
             len_photo_ds = len_photo_ds + len(i)
         len_photo_ds
```

Out[16]: 288

```
In [17]: # monet_ds.cardinality().numpy()
         #INFINITE_CARDINALITY    -1
         #UNKNOWN_CARDINALITY     -2
```

4. Model preparation

In order to build our GAN, we use the DCGAN tutorial from TensorFlow helping us to prepare all the functions we need:

<https://www.tensorflow.org/tutorials/generative/dcgan?hl=en>

As this tutorial used a CNN architecture which is not adapted to our problem, we decide to try different architectures that we found on the net.

4.1. Different type of Neural Networks

GANs, and specially computer vision in machine learning use a type of neural networks called Convolutional Neural Networks. CNN are very good to understand different types of features in image processing. The two NN we will focus use these types of NN:

- CNN-based model that proved good results with a lot of epochs.
- CNN based on U-Net model.

The CNN-based scratch model is based on

<https://www.geeksforgeeks.org/building-a-generative-adversarial-network-using-keras/> made in 2019/06.

It uses functions and methods which are not really fitted anymore for GAN use (Upsampling2D, relu, ...) but seems to give good results for generating images.

The second one will use methods from pix2pix CNN based on

[https://www.tensorflow.org/tutorials/generative/pix2pix?](https://www.tensorflow.org/tutorials/generative/pix2pix?hl=en#build_the_generator)

[hl=en#build_the_generator](https://www.tensorflow.org/tutorials/generative/pix2pix?hl=en#build_the_generator) Tensorflow tutorial in 2022/08. It usually uses a simplified U-Net architecture. It's the same used in the CycleGAN tutorial, without the skip layers because we will generate images from a random vector, not from an input image.

Let's start by designing the generator and discriminator. Then we will build the GAN architecture.

```
In [18]: input_shape = tuple(IMAGE_SIZE) + (3,) #[256,256,3]
         input_shape
```

```
Out[18]: (256, 256, 3)
```

4.1.1. Generator design

The first generator design based from the link of GeekForGeek looks like that:

```
In [19]: def basic_generator():

         #inputs = keras.Input(shape=input_shape)
         model = tf.keras.Sequential()

         #Building the input layer
         model.add(keras.Input(shape=(100,)))

         model.add(layers.Dense(128 * 32 * 32, activation="relu"))
         model.add(layers.Reshape((32, 32, 128)))

         #print(model.output_shape)
         model.add(layers.UpSampling2D())
         #print(model.output_shape)

         model.add(layers.Conv2D(256, kernel_size=3, padding="same"))
         model.add(layers.BatchNormalization(momentum=0.78))
         model.add(layers.LeakyReLU())

         model.add(layers.UpSampling2D())

         model.add(layers.Conv2D(128, kernel_size=3, padding="same"))
         model.add(layers.BatchNormalization(momentum=0.78))
         model.add(layers.LeakyReLU())

         #print(model.output_shape)
         model.add(layers.UpSampling2D())

         #print(model.output_shape)
```



```

model.add(layers.Conv2D(64, kernel_size=3, padding="same"))
model.add(layers.BatchNormalization(momentum=0.78))
model.add(layers.LeakyReLU())

#print(model.output_shape)

model.add(layers.Conv2D(3, kernel_size=3, padding="same"))
model.add(layers.Activation("relu"))

#assert model.output_shape == (None, 28, 28, 1) # we need to
#model.add(layers.Dense(3, activation="relu"))
"""
last_input = model.output_shape
print("last_input: " + str(last_input))

k_size = last_input[1] - img_sz*np.sign(last_input[1] - img_s

print("k_size: "+str(k_size))
print(str(np.sign(last_input[1] - img_sz)))
model.add(layers.Conv2D(3, kernel_size=(k_size,k_size), paddi
model.add(layers.Activation("tanh"))
"""
#print(model.output_shape)
assert model.output_shape == (None, img_sz, img_sz, 3) # we n

#Generating the output image
#noise = Input(shape=(latent_dimensions,))
#image = model(noise)

#return Model(noise, image)
return model

```

It's a relatively simple design with few layers.

The second one is based on a U-NET architecture, an efficient architecture used for these types of problems. It's based on the generator based on https://www.tensorflow.org/tutorials/generative/pix2pix?hl=en#build_the_generator and it's quite the same as the one from the CycleGAN tutorial except the skips layers that are used if we try to change the nature of an existing image. But we are generating a new image from a random vector, so these layers should not be needed!

U-Net is a convolutional neural network architecture designed for image segmentation tasks, which involves dividing an image into different regions or objects of interest. The U-Net architecture consists of an encoder and a decoder, which are connected by a bridge that passes information between the two parts of the network. The encoder is made up of several convolutional layers that extract features from the input image, while the decoder uses up-sampling layers to increase the resolution of the feature maps and generate a segmentation map. The architecture also includes skip connections that allow the decoder to use information from earlier layers in the encoder to generate more accurate segmentation maps. The U-Net architecture has been shown to perform well on various medical image segmentation tasks, including segmentation of organs and tumors. An illustration of the U-Net architecture can be found at this URL: (<https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/u-net-architecture.png>)

In [20]: `OUTPUT_CHANNELS = 3`

In [21]: `def downsample(filters, size, apply_batchnorm=True):`

```

initializer = tf.random_normal_initializer(0., 0.02)

result = tf.keras.Sequential()
result.add(
    tf.keras.layers.Conv2D(filters, size, strides=2, padding='same',
                           kernel_initializer=initializer, use_bias=False))

if apply_batchnorm:
    result.add(tf.keras.layers.BatchNormalization())

result.add(tf.keras.layers.LeakyReLU())

return result

```

```

In [22]: inp = example_photo[0] * 0.5 + 0.5

down_model = downsample(3, 4)
down_result = down_model(tf.expand_dims(inp, 0))
print (down_result.shape)

```

(1, 128, 128, 3)

```

In [23]: def upsample(filters, size, apply_dropout=False):
        initializer = tf.random_normal_initializer(0., 0.02)

        result = tf.keras.Sequential()
        result.add(
            tf.keras.layers.Conv2DTranspose(filters, size, strides=2,
                                             padding='same',
                                             kernel_initializer=initializer,
                                             use_bias=False))

        result.add(tf.keras.layers.BatchNormalization())

        if apply_dropout:
            result.add(tf.keras.layers.Dropout(0.5))

        result.add(tf.keras.layers.ReLU())

        return result

```

```

In [24]: up_model = upsample(3, 4)
up_result = up_model(down_result)
print (up_result.shape)

```

(1, 256, 256, 3)

```

In [25]: def pix_generator():

    #inputs = keras.Input(shape=input_shape)

    #Building the input layer

    #inputs = tf.keras.layers.Input(shape=[256, 256, 3])

    inputs = tf.keras.layers.Input(shape=(100,))

    x = inputs
    x = layers.Dense(256 * 256 * 3, activation="relu") (x)
    x = layers.Reshape((256, 256, 3))(x)

    down_stack = [
        downsample(64, 4, apply_batchnorm=False), # (batch_size, 128
        downsample(128, 4), # (batch_size, 64, 64, 128)
        downsample(256, 4), # (batch_size, 32, 32, 256)
        downsample(512, 4), # (batch_size, 16, 16, 512)
        downsample(512, 4), # (batch_size, 8, 8, 512)
        downsample(512, 4), # (batch_size, 4, 4, 512)
        downsample(512, 4), # (batch_size, 2, 2, 512)
        downsample(512, 4), # (batch_size, 1, 1, 512)
    ]

```

```

]

up_stack = [
    upsample(512, 4, apply_dropout=True), # (batch_size, 2, 2, 1
    upsample(512, 4, apply_dropout=True), # (batch_size, 4, 4, 1
    upsample(512, 4, apply_dropout=True), # (batch_size, 8, 8, 1
    upsample(512, 4), # (batch_size, 16, 16, 1024)
    upsample(256, 4), # (batch_size, 32, 32, 512)
    upsample(128, 4), # (batch_size, 64, 64, 256)
    upsample(64, 4), # (batch_size, 128, 128, 128)
]

initializer = tf.random_normal_initializer(0., 0.02)
last = tf.keras.layers.Conv2DTranspose(OUTPUT_CHANNELS, 4,
                                       strides=2,
                                       padding='same',
                                       kernel_initializer=initializer,
                                       activation='tanh') # (batch_size,

#x = inputs
"""
# Downsampling through the model
skips = []
for down in down_stack:
    x = down(x)
    skips.append(x)

skips = reversed(skips[:-1])

# Upsampling and establishing the skip connections
for up, skip in zip(up_stack, skips):
    x = up(x)
    x = tf.keras.layers.Concatenate()([x, skip])
"""

# Downsampling through the model
for down in down_stack:
    x = down(x)

# Upsampling and establishing the skip connections
for up in up_stack:
    x = up(x)

x = last(x)

return tf.keras.Model(inputs=inputs, outputs=x, name="pix_generator")

```

In [26]:

```

def dcgan_generator():

    #inputs = keras.Input(shape=input_shape)

    #Building the input Layer

    #inputs = tf.keras.layers.Input(shape=[256, 256, 3])

    inputs = tf.keras.layers.Input(shape=(100,))

    x = inputs
    x = layers.Dense(4 * 4 * 4096, activation="relu")(x)
    x = layers.Reshape((4, 4, 4096))(x)

    up_stack = [
        upsample(2048, 5, apply_dropout=True), # (batch_size, 8, 8,
        upsample(1024, 5, apply_dropout=True), # (batch_size, 16, 16,
        upsample(512, 5, apply_dropout=True), # (batch_size, 64, 64,
        upsample(256, 5), # (batch_size, 128, 128, 256)
        upsample(128, 5), # (batch_size, 256, 256, 128)
    ]

    initializer = tf.random_normal_initializer(0., 0.02)
    last = tf.keras.layers.Conv2DTranspose(OUTPUT_CHANNELS, 5,
                                       strides=2,

```

```

        strides=2,
        padding='same',
        kernel_initializer=initializer,
        activation='tanh') # (batch_size, 256, 256, 3)

#x = inputs

# Upsampling and establishing the skip connections
for up in up_stack:
    x = up(x)

x = last(x)

return tf.keras.Model(inputs=inputs, outputs=x, name="dcgan_generator_2")

```

In [27]:

```

"""
def dcgan_generator_2():

    #inputs = keras.Input(shape=input_shape)

    #Building the input layer

    #inputs = tf.keras.layers.Input(shape=[256, 256, 3])

    inputs = tf.keras.layers.Input(shape=(100,))

    x = inputs
    x = layers.Dense(8 * 8 * 2048, activation="relu")(x)
    x = layers.Reshape((8, 8, 2048))(x)

    up_stack = [
        #upsample(2048, 5, apply_dropout=True), # (batch_size, 8, 8, 2048)
        upsample(1024, 5, apply_dropout=True), # (batch_size, 16, 16, 1024)
        upsample(512, 5, apply_dropout=True), # (batch_size, 64, 64, 512)
        upsample(256, 5), # (batch_size, 128, 128, 256)
        upsample(128, 5), # (batch_size, 256, 256, 128)
    ]

    initializer = tf.random_normal_initializer(0., 0.02)
    last = tf.keras.layers.Conv2DTranspose(OUTPUT_CHANNELS, 5,
                                           strides=2,
                                           padding='same',
                                           kernel_initializer=initializer,
                                           activation='tanh') # (batch_size, 256, 256, 3)

    #x = inputs

    # Upsampling and establishing the skip connections
    for up in up_stack:
        x = up(x)

    x = last(x)

    return tf.keras.Model(inputs=inputs, outputs=x, name="dcgan_generator_2")

```

Out[27]: '\ndef dcgan_generator_2():\n \n #inputs = keras.Input(shape=input_shape)\n \n #Building the input layer\n \n #inputs = tf.keras.layers.Input(shape=[256, 256, 3])\n \n inputs = tf.keras.layers.Input(shape=(100,))\n \n x = inputs\n x = layers.Dense(8 * 8 * 2048, activation="relu")(x)\n x = layers.Reshape((8, 8, 2048))(x)\n \n up_stack = [\n #upsample(2048, 5, apply_dropout=True), # (batch_size, 8, 8, 2048)\n upsample(1024, 5, apply_dropout=True), # (batch_size, 16, 16, 1024)\n upsample(512, 5, apply_dropout=True), # (batch_size, 64, 64, 512)\n upsample(256, 5), # (batch_size, 128, 128, 256)\n upsample(128, 5), # (batch_size, 256, 256, 128)\n]\n \n initializer = tf.random_normal_initializer(0., 0.02)\n last = tf.keras.layers.Conv2DTranspose(OUTPUT_CHANNELS, 5,\n strides=2,\n padding='same',\n kernel_initializer=initializer,\n activation='tanh') # (batch_size, 256, 256, 3)\n \n #x = inputs

```

\n\n\n    # Upsampling and establishing the skip connections\n    for
up in up_stack:\n        x = up(x)\n        x = last(x)\n        return t
f.keras.Model(inputs=inputs, outputs=x, name="dcgan_generator_2")'

```

In [28]:

```
def dcgan_generator_2():

    #inputs = keras.Input(shape=input_shape)

    #Building the input layer

    #inputs = tf.keras.layers.Input(shape=[256, 256, 3])

    inputs = tf.keras.layers.Input(shape=(100,))

    x = inputs
    x = layers.Dense(8 * 8 * 1024, activation="relu") (x)
    x = layers.Reshape((8, 8, 1024))(x)

    up_stack = [
        #upsample(2048, 5, apply_dropout=True), # (batch_size, 8, 8,
        upsample(512, 5, apply_dropout=True), # (batch_size, 16, 16,
        upsample(256, 5, apply_dropout=True), # (batch_size, 64, 64,
        upsample(128, 5), # (batch_size, 128, 128, 128)
        upsample(256, 5), # (batch_size, 256, 256, 64)
    ]

    initializer = tf.random_normal_initializer(0., 0.02)
    last = tf.keras.layers.Conv2DTranspose(OUTPUT_CHANNELS, 5,
                                           strides=2,
                                           padding='same',
                                           kernel_initializer=initializer,
                                           activation='tanh') # (batch_size, 256, 256, 3)

    #x = inputs

    # Upsampling and establishing the skip connections
    for up in up_stack:
        x = up(x)

    x = last(x)

    return tf.keras.Model(inputs=inputs, outputs=x, name="dcgan_generator_2")
```

In [29]:

```
def dcgan_generator_3():

    #inputs = keras.Input(shape=input_shape)

    #Building the input layer

    #inputs = tf.keras.layers.Input(shape=[256, 256, 3])

    inputs = tf.keras.layers.Input(shape=(100,))

    x = inputs
    x = layers.Dense(8 * 8 * 512, activation="relu") (x)
    x = layers.Reshape((8, 8, 512))(x)

    up_stack = [
        #upsample(2048, 5, apply_dropout=True), # (batch_size, 8, 8,
        upsample(256, 5, apply_dropout=True), # (batch_size, 16, 16,
        upsample(128, 5, apply_dropout=True), # (batch_size, 64, 64,
        upsample(64, 5), # (batch_size, 128, 128, 128)
        upsample(32, 5), # (batch_size, 256, 256, 64)
    ]

    initializer = tf.random_normal_initializer(0., 0.02)
    last = tf.keras.layers.Conv2DTranspose(OUTPUT_CHANNELS, 5,
                                             strides=2,
                                             padding='same',
                                             kernel_initializer=initializer,
                                             activation='tanh') # (batch_size, 256, 256, 3)
```

```

#x = inputs

# Upsampling and establishing the skip connections
for up in up_stack:
    x = up(x)

x = last(x)

return tf.keras.Model(inputs=inputs, outputs=x, name="dcgan_generator")

```

In [30]:

```

gan_gen = dcgan_generator_3()
gan_gen.summary()

```

Model: "dcgan_generator_3"

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 100)]	0
dense (Dense)	(None, 32768)	3309568
reshape (Reshape)	(None, 8, 8, 512)	0
sequential_2 (Sequential)	(None, 16, 16, 256)	3277824
sequential_3 (Sequential)	(None, 32, 32, 128)	819712
sequential_4 (Sequential)	(None, 64, 64, 64)	205056
sequential_5 (Sequential)	(None, 128, 128, 32)	51328
conv2d_transpose_5 (Conv2DTr	(None, 256, 256, 3)	2403
Total params: 7,665,891		
Trainable params: 7,664,931		
Non-trainable params: 960		

In [31]:

```

# Exemple of generated images from a photo input passed through ou ba
noise_input = tf.random.normal([1, 100])

for test_generator in [basic_generator(),pix_generator(), dcgan_generator_3()]:
    test_generator.summary()

    start = time.time()
    to_monet = test_generator(noise_input)
    print ('Time for one generated_image from photo is {} sec'.format(time.time() - start))

    plt.subplot(1, 2, 1)
    plt.title("Original Photo")
    plt.imshow(example_photo[0] * 0.5 + 0.5)

    plt.subplot(1, 2, 2)
    plt.title("Monet-esque Photo")
    plt.imshow(to_monet[0] * 0.5 + 0.5)
    plt.show()

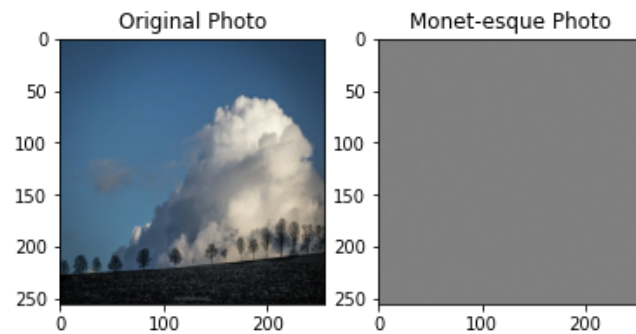
```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 131072)	13238272
reshape_1 (Reshape)	(None, 32, 32, 128)	0
up_sampling2d (UpSampling2D)	(None, 64, 64, 128)	0
conv2d_1 (Conv2D)	(None, 64, 64, 256)	295168
batch normalization_6 (Batch	(None, 64, 64, 256)	1024

leaky_re_lu_1 (LeakyReLU)	(None, 64, 64, 256)	0
up_sampling2d_1 (UpSampling2D)	(None, 128, 128, 256)	0
conv2d_2 (Conv2D)	(None, 128, 128, 128)	295040
batch_normalization_7 (Batch Normalization)	(None, 128, 128, 128)	512
leaky_re_lu_2 (LeakyReLU)	(None, 128, 128, 128)	0
up_sampling2d_2 (UpSampling2D)	(None, 256, 256, 128)	0
conv2d_3 (Conv2D)	(None, 256, 256, 64)	73792
batch_normalization_8 (Batch Normalization)	(None, 256, 256, 64)	256
leaky_re_lu_3 (LeakyReLU)	(None, 256, 256, 64)	0
conv2d_4 (Conv2D)	(None, 256, 256, 3)	1731
activation (Activation)	(None, 256, 256, 3)	0
=====		
Total params: 13,905,795		
Trainable params: 13,904,899		
Non-trainable params: 896		

Time for one generated_image from photo is 0.27823781967163086 sec

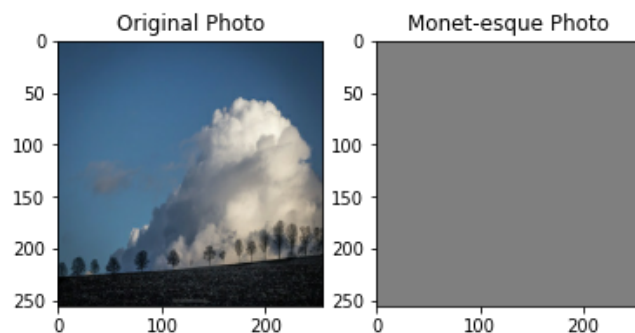


Model: "pix_generator"

Layer (type)	Output Shape	Param #
=====		
input_3 (InputLayer)	[(None, 100)]	0
dense_2 (Dense)	(None, 196608)	19857408
reshape_2 (Reshape)	(None, 256, 256, 3)	0
sequential_7 (Sequential)	(None, 128, 128, 64)	3072
sequential_8 (Sequential)	(None, 64, 64, 128)	131584
sequential_9 (Sequential)	(None, 32, 32, 256)	525312
sequential_10 (Sequential)	(None, 16, 16, 512)	2099200
sequential_11 (Sequential)	(None, 8, 8, 512)	4196352
sequential_12 (Sequential)	(None, 4, 4, 512)	4196352
sequential_13 (Sequential)	(None, 2, 2, 512)	4196352
sequential_14 (Sequential)	(None, 1, 1, 512)	4196352
sequential_15 (Sequential)	(None, 2, 2, 512)	4196352
sequential_16 (Sequential)	(None, 4, 4, 512)	4196352
sequential_17 (Sequential)	(None, 8, 8, 512)	4196352
sequential_18 (Sequential)	(None, 16, 16, 512)	4196352

sequential_19 (Sequential)	(None, 32, 32, 256)	2098176
sequential_20 (Sequential)	(None, 64, 64, 128)	524800
sequential_21 (Sequential)	(None, 128, 128, 64)	131328
conv2d_transpose_13 (Conv2DT	(None, 256, 256, 3)	3075
=====		
Total params: 58,944,771		
Trainable params: 58,933,891		
Non-trainable params: 10,880		

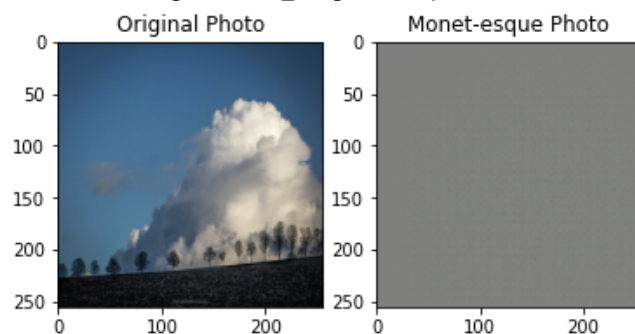
Time for one generated_image from photo is 0.19965362548828125 sec



Model: "drgan_generator"

Layer (type)	Output Shape	Param #
=====		
input_4 (InputLayer)	[(None, 100)]	0
dense_3 (Dense)	(None, 65536)	6619136
reshape_3 (Reshape)	(None, 4, 4, 4096)	0
sequential_22 (Sequential)	(None, 8, 8, 2048)	209723392
sequential_23 (Sequential)	(None, 16, 16, 1024)	52432896
sequential_24 (Sequential)	(None, 32, 32, 512)	13109248
sequential_25 (Sequential)	(None, 64, 64, 256)	3277824
sequential_26 (Sequential)	(None, 128, 128, 128)	819712
conv2d_transpose_19 (Conv2DT	(None, 256, 256, 3)	9603
=====		
Total params: 285,991,811		
Trainable params: 285,983,875		
Non-trainable params: 7,936		

Time for one generated_image from photo is 0.6102311611175537 sec



Model: "drgan_generator_2"

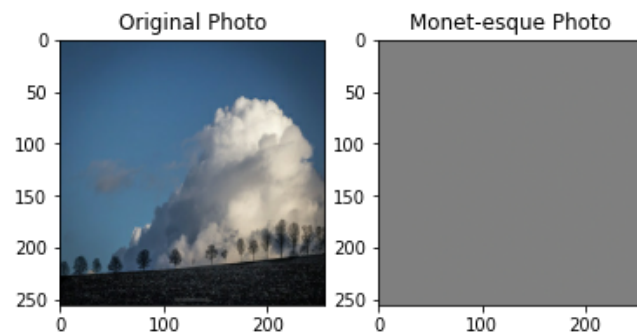
Layer (type)	Output Shape	Param #
=====		
input_5 (InputLayer)	[(None, 100)]	0
dense_4 (Dense)	(None, 65536)	6619136
reshape_4 (Reshape)	(None, 8, 8, 1024)	0


```

reshape_4 (Reshape)      (None, 8, 8, 1024)      0
-----
sequential_27 (Sequential) (None, 16, 16, 512)      13109248
-----
sequential_28 (Sequential) (None, 32, 32, 256)      3277824
-----
sequential_29 (Sequential) (None, 64, 64, 128)      819712
-----
sequential_30 (Sequential) (None, 128, 128, 256)    820224
-----
conv2d_transpose_24 (Conv2DT (None, 256, 256, 3)      19203
=====
Total params: 24,665,347
Trainable params: 24,663,043
Non-trainable params: 2,304

```

Time for one generated_image from photo is 0.17995882034301758 sec



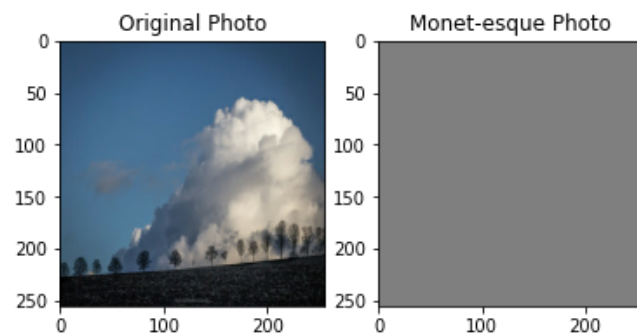
Model: "drgan_generator_3"

```

Layer (type)              Output Shape              Param #
-----
input_6 (InputLayer)      [(None, 100)]             0
-----
dense_5 (Dense)           (None, 32768)             3309568
-----
reshape_5 (Reshape)       (None, 8, 8, 512)         0
-----
sequential_31 (Sequential) (None, 16, 16, 256)        3277824
-----
sequential_32 (Sequential) (None, 32, 32, 128)        819712
-----
sequential_33 (Sequential) (None, 64, 64, 64)         205056
-----
sequential_34 (Sequential) (None, 128, 128, 32)       51328
-----
conv2d_transpose_29 (Conv2DT (None, 256, 256, 3)       2403
=====
Total params: 7,665,891
Trainable params: 7,664,931
Non-trainable params: 960

```

Time for one generated_image from photo is 0.04739117622375488 sec



In []:

We need to generate at least 7000 images in less than 5h (and 3h if we use TPU).

Let see the average calculation for 100 images generated for each methods

Let see the average calculation for 100 images generated for each methods.

```
In [32]: # Exemple of generated images from a photo input passed through our b

gen_time = [[],[],[],[],[ ]]
for num_ex in range(100):

    noise_input = tf.random.normal([1, 100])
    i=0
    for test_generator in [basic_generator(),pix_generator(), dcgan_g
        #test_generator.summary()

        start = time.time()
        to_monet = test_generator(noise_input)
        #print ('Time for one generated_image from photo is {} sec'.f

        time_gen = time.time()-start
        gen_time[i].append(time_gen)
        i+=1
```

```
In [33]: len(gen_time)
```

Out[33]: 5

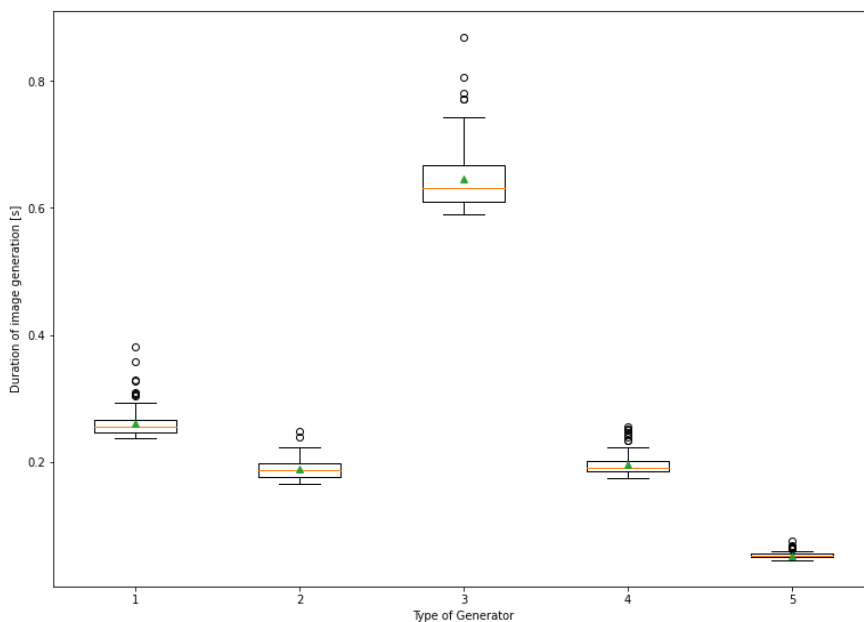
```
In [34]: len(gen_time[0])
```

Out[34]: 100

```
In [35]: fig = plt.figure(figsize =(10, 7))

# Creating axes instance
ax = fig.add_axes([0, 0, 1, 1])

# Creating plot
bp = ax.boxplot(gen_time, showmeans=True)
plt.xlabel("Type of Generator")
plt.ylabel("Duration of image generation [s]")
#plt.legend(['Duration', 'Mean'])
#ax.legend([bp['boxes'][0],bp['boxes'][1],bp['boxes'][2]],['A', 'B', 'C'])
# show plot
plt.show()
```



```
In [36]: fig = plt.figure(figsize =(10, 7))

for i in range(len(gen_time)):
```

```

plt.plot(gen_time[i])

#plt.plot(np.mean(gen_time[i]),marker='o')

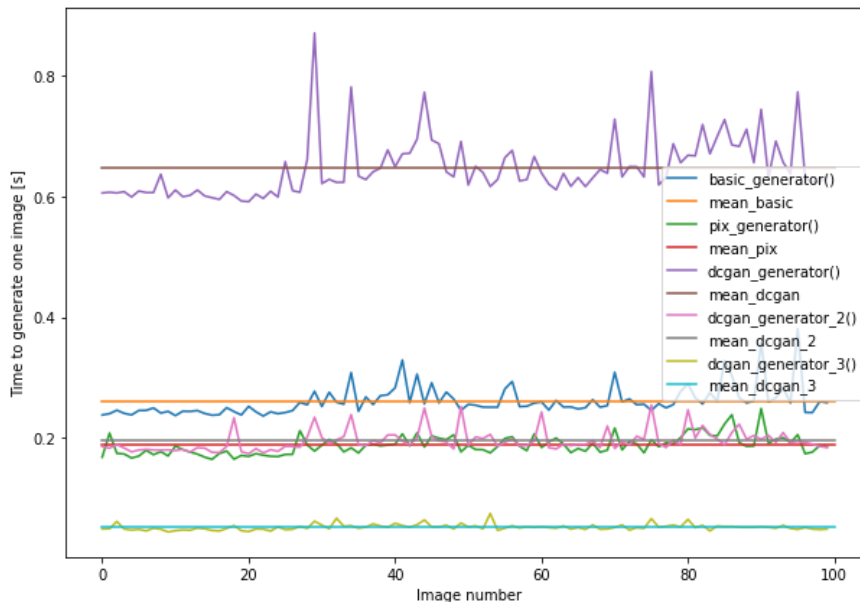
plt.plot([0,100],[np.mean(gen_time[i]),np.mean(gen_time[i])])

plt.legend(['basic_generator()', 'mean_basic', 'pix_generator()', 'mean_

plt.xlabel("Image number")
plt.ylabel("Time to generate one image [s]")

plt.show()

```



```

In [37]: """
fig = plt.figure(figsize =(10, 7))

plt.plot(gen_time[0])
plt.plot(gen_time[1])

plt.plot(np.mean(gen_time[0]),marker='o')
plt.plot(np.mean(gen_time[1]),marker='o')

plt.plot([0,100],[np.mean(gen_time[0]),np.mean(gen_time[0])])
plt.plot([0,100],[np.mean(gen_time[1]),np.mean(gen_time[1])])

plt.legend(['basic_generator()', 'pix_generator()',
            'mean_', 'mean_pix'])

plt.xlabel("Image number")
plt.ylabel("Time to generate one image [s]")

plt.show()"""

```

```

Out[37]: '\nfig = plt.figure(figsize =(10, 7))\n\nplt.plot(gen_time[0])\nplt.p
lot(gen_time[1])\n\nplt.plot(np.mean(gen_time[0]),marker=\ 'o\ ')\nplt.
plot(np.mean(gen_time[1]),marker=\ 'o\ ')\n\nplt.plot([0,100],[np.mean
(gen_time[0]),np.mean(gen_time[0])])\nplt.plot([0,100],[np.mean(gen_t
ime[1]),np.mean(gen_time[1])])\n\nplt.legend([\ 'basic_generator()
\ ',\ 'pix_generator()'\ ,\n            \ 'mean_\ ',\ 'mean_pix\ '])\n\nplt.x
label("Image number")\nplt.ylabel("Time to generate one image [s]")\n\n
\nplt.show()'

```

1h = 3600s . The average time for geenrate images without TPU is 0.25s for pix_generator.

--> 3600s/0.25s = 14400 images hoicould be generated in one hour.

We need to generate at least 7000 images in less than 5h (3h if we use TPU).

This method seems computationnaly valuable. Let's test it.

In average, the U-net based pix_generator seems quite faster than the basic generator to generate images from an input vector. We choose to continue with the fastest.

The second method using a U-Net-like architecture seems faster to generate images.

```
In [38]: input_shape
```

```
Out[38]: (256, 256, 3)
```

```
In [39]: example_photo.shape
```

```
Out[39]: TensorShape([32, 256, 256, 3])
```

```
In [40]: example_photo[0].shape
```

```
Out[40]: TensorShape([256, 256, 3])
```

```
In [41]: noise_input.shape
```

```
Out[41]: TensorShape([1, 100])
```

4.1.2. Discriminator design

We build two different types of discriminators. But based on the study made previously with geenrators, we decide to keep the discriminator used with U-net architecture geenrator: pix_discriminator.

```
In [42]: def basic_discriminator():
#Building the convolutional layers
#to classify whether an image is real or fake

#inputs = tf.keras.Input(shape=input_shape)
model = tf.keras.Sequential()

#Building the input layer
model.add(keras.Input(shape=input_shape))

#model = Sequential()

model.add(layers.Conv2D(32, kernel_size=3, strides=2, padding="sa
model.add(layers.LeakyReLU(alpha=0.2))
model.add(layers.Dropout(0.25))

model.add(layers.Conv2D(64, kernel_size=3, strides=2, padding="sa
model.add(layers.ZeroPadding2D(padding=((0,1),(0,1))))
model.add(layers.BatchNormalization(momentum=0.82))
model.add(layers.LeakyReLU(alpha=0.25))
model.add(layers.Dropout(0.25))

model.add(layers.Conv2D(128, kernel_size=3, strides=2, padding="s
model.add(layers.BatchNormalization(momentum=0.82))
model.add(layers.LeakyReLU(alpha=0.2))
model.add(layers.Dropout(0.25))

model.add(layers.Conv2D(256, kernel_size=3, strides=1, padding="s
model.add(layers.BatchNormalization(momentum=0.8))
model.add(layers.LeakyReLU(alpha=0.25))
model.add(layers.Dropout(0.25))

#print(model.output_shape)
```

```

#Building the output layer
model.add(layers.Flatten())
print(model.output_shape)
#assert model.output_shape == (None, 278784)
model.add(layers.Dense(1, activation='sigmoid'))

#image = Input(shape=image_shape)
#validity = model(image)

#return Model(image, validity)
return model

```

In [43]:

```

def pix_discriminator():
    initializer = tf.random_normal_initializer(0., 0.02)

    #inp = tf.keras.layers.Input(shape=[256, 256, 3], name='input_image')
    #tar = tf.keras.layers.Input(shape=[256, 256, 3], name='target_image')

    #x = tf.keras.layers.concatenate([inp, tar]) # (batch_size, 256,
    # 256, 6)

    inputs = tf.keras.layers.Input(shape=input_shape)

    x = inputs

    down1 = downsample(64, 4, False)(x) # (batch_size, 128, 128, 64)
    down2 = downsample(128, 4)(down1) # (batch_size, 64, 64, 128)
    down3 = downsample(256, 4)(down2) # (batch_size, 32, 32, 256)

    zero_pad1 = tf.keras.layers.ZeroPadding2D()(down3) # (batch_size,
    # 32, 32, 256, 1)
    conv = tf.keras.layers.Conv2D(512, 4, strides=1,
                                   kernel_initializer=initializer,
                                   use_bias=False)(zero_pad1) # (batch_
    # size, 28, 28, 512)

    batchnorm1 = tf.keras.layers.BatchNormalization()(conv)

    leaky_relu = tf.keras.layers.LeakyReLU()(batchnorm1)

    zero_pad2 = tf.keras.layers.ZeroPadding2D()(leaky_relu) # (batch_
    # size, 28, 28, 512, 1)

    last = tf.keras.layers.Conv2D(1, 4, strides=1,
                                   kernel_initializer=initializer)(zero_
    # pad2) # (batch_size, 28, 28, 1)

    last2 = tf.keras.layers.Flatten()(last)
    last3 = tf.keras.layers.Dense(1)(last2)

    return tf.keras.Model(inputs=inputs, outputs=last3, name="pix_dis

```

In [44]:

```

def dcgan_discriminator():
    initializer = tf.random_normal_initializer(0., 0.02)

    #inp = tf.keras.layers.Input(shape=[256, 256, 3], name='input_image')
    #tar = tf.keras.layers.Input(shape=[256, 256, 3], name='target_image')

    #x = tf.keras.layers.concatenate([inp, tar]) # (batch_size, 256,
    # 256, 6)

    inputs = tf.keras.layers.Input(shape=input_shape)

    x = inputs

    down_stack = [
        downsample(256, 5, apply_batchnorm=False), # (batch_size, 12
        downsample(512, 5), # (batch_size, 64, 64, 512)
        downsample(1024, 5), # (batch_size, 32, 32, 1024)
        downsample(2048, 5), # (batch_size, 16, 16, 2048)
        downsample(4096, 5), # (batch_size, 8, 8, 4096)
        #downsample(8192, 5), # (batch_size, 4, 4, 8192) #too much c
    ]

    # Downsampling through the model

    for down in down_stack:
        x = down(x)

```

```
last = tf.keras.layers.Conv2D(1, 5, strides=2,
                               kernel_initializer=initializer)(x)

last2 = tf.keras.layers.Flatten()(last)
last3 = tf.keras.layers.Dense(1, activation='sigmoid')(last2)

return tf.keras.Model(inputs=inputs, outputs=last3, name="dcgan_d")
```

In [45]:

```
def dcgan_discriminator_2():
    initializer = tf.random_normal_initializer(0., 0.02)

    #inp = tf.keras.layers.Input(shape=[256, 256, 3], name='input_image')
    #tar = tf.keras.layers.Input(shape=[256, 256, 3], name='target_image')

    #x = tf.keras.layers.concatenate([inp, tar]) # (batch_size, 256, 256, 6)

    inputs = tf.keras.layers.Input(shape=input_shape)

    x = inputs

    down_stack = [
        downsample(256, 5, apply_batchnorm=False), # (batch_size, 128, 128, 5)
        downsample(512, 5), # (batch_size, 64, 64, 512)
        downsample(1024, 5), # (batch_size, 32, 32, 1024)
        #downsample(2048, 5), # (batch_size, 16, 16, 2048)
        #downsample(4096, 5), # (batch_size, 8, 8, 4096)
        #downsample(8192, 5), # (batch_size, 4, 4, 8192) #too much computation
    ]

    # Downsampling through the model
    for down in down_stack:
        x = down(x)

    last = tf.keras.layers.Conv2D(1, 5, strides=2,
                                   kernel_initializer=initializer)(x)

    last2 = tf.keras.layers.Flatten()(last)
    last3 = tf.keras.layers.Dense(1, activation='sigmoid')(last2)

    return tf.keras.Model(inputs=inputs, outputs=last3, name="dcgan_discriminator_2")
```

In [46]:

[illegible]

```

last2 = tf.keras.layers.Flatten()(last)
last3 = tf.keras.layers.Dense(1, activation='sigmoid')(last2)

return tf.keras.Model(inputs=inputs, outputs=last3, name="dcgan_d

```

```

In [47]: disc = dcgan_discriminator_2()
np.array_str(disc(example_monet).numpy())

```

```

Out[47]: '[[0.5166924 ]\n [0.5108497 ]\n [0.51678234]\n [0.78177786]\n [0.6231
432 ]\n [0.6472901 ]\n [0.4875572 ]\n [0.64755166]\n [0.6628911 ]\n
[0.43695906]\n [0.47369492]\n [0.59492934]\n [0.47127798]\n [0.570579
5 ]\n [0.5345459 ]\n [0.6230628 ]\n [0.6736251 ]\n [0.66989005]\n [0.
75696856]\n [0.5105291 ]\n [0.47339135]\n [0.41122985]\n [0.6595695 ]
\n [0.55888426]\n [0.6434647 ]\n [0.5897669 ]\n [0.51017964]\n [0.535
9782 ]\n [0.52674985]\n [0.7142397 ]\n [0.38711327]\n [0.6232749 ]]'

```

```

In [48]: # Too much memory usage
plt.subplot(1, 2, 1)
plt.title("Original Photo")
plt.imshow(example_monet[0] * 0.5 + 0.5)

for test_discriminator in [basic_discriminator(), pix_discriminator()],
    test_discriminator.summary()

decision_monet = test_discriminator(example_monet)
#decision_to_monet = test_discriminator(to_monet)
print ("decision_monet: " + np.array_str(decision_monet.numpy()))
decision_monet
#print ("decision_to_monet: " + decision_to_monet)

```

(None, 278784)
Model: "sequential_2938"

Layer (type)	Output Shape	Param #
conv2d_1217 (Conv2D)	(None, 128, 128, 32)	896
leaky_re_lu_1115 (LeakyReLU)	(None, 128, 128, 32)	0
dropout_1012 (Dropout)	(None, 128, 128, 32)	0
conv2d_1218 (Conv2D)	(None, 64, 64, 64)	18496
zero_padding2d (ZeroPadding2D)	(None, 65, 65, 64)	0
batch_normalization_3038 (Batch Normalization)	(None, 65, 65, 64)	256
leaky_re_lu_1116 (LeakyReLU)	(None, 65, 65, 64)	0
dropout_1013 (Dropout)	(None, 65, 65, 64)	0
conv2d_1219 (Conv2D)	(None, 33, 33, 128)	73856
batch_normalization_3039 (Batch Normalization)	(None, 33, 33, 128)	512
leaky_re_lu_1117 (LeakyReLU)	(None, 33, 33, 128)	0
dropout_1014 (Dropout)	(None, 33, 33, 128)	0
conv2d_1220 (Conv2D)	(None, 33, 33, 256)	295168
batch_normalization_3040 (Batch Normalization)	(None, 33, 33, 256)	1024
leaky_re_lu_1118 (LeakyReLU)	(None, 33, 33, 256)	0
dropout_1015 (Dropout)	(None, 33, 33, 256)	0
flatten_1 (Flatten)	(None, 278784)	0
dense_507 (Dense)	(None, 1)	278785

Total params: 668,993
Trainable params: 668,097
Non-trainable params: 896

decision_monet: [[0.4989608]
[0.50033855]
[0.4892774]
[0.49656895]
[0.49945757]
[0.4945874]
[0.493519]
[0.50581294]
[0.4958876]
[0.49877587]
[0.4939986]
[0.50502]
[0.49647]
[0.49209636]
[0.49759874]
[0.49887758]
[0.49787822]
[0.48983026]
[0.49454442]
[0.49546227]
[0.49122837]
[0.49816656]
[0.49734196]
[0.49450192]
[0.49282676]
[0.50356746]
[0.50135475]
[0.49768695]
[0.49209216]
[0.4890001]
[0.49633172]
[0.49915007]]

Model: "pix_discriminator"

Layer (type)	Output Shape	Param #
=====		
input_509 (InputLayer)	[(None, 256, 256, 3)]	0

sequential_2939 (Sequential)	(None, 128, 128, 64)	3072

sequential_2940 (Sequential)	(None, 64, 64, 128)	131584

sequential_2941 (Sequential)	(None, 32, 32, 256)	525312

zero_padding2d_1 (ZeroPaddin	(None, 34, 34, 256)	0

conv2d_1224 (Conv2D)	(None, 31, 31, 512)	2097152

batch_normalization_3043 (Ba	(None, 31, 31, 512)	2048

leaky_re_lu_1122 (LeakyReLU)	(None, 31, 31, 512)	0

zero_padding2d_2 (ZeroPaddin	(None, 33, 33, 512)	0

conv2d_1225 (Conv2D)	(None, 30, 30, 1)	8193

flatten_2 (Flatten)	(None, 900)	0

dense_508 (Dense)	(None, 1)	901
=====		
Total params: 2,768,262		
Trainable params: 2,766,470		
Non-trainable params: 1,792		

decision_monet: [[-0.03048379]
[-0.0085839]
[-0.01148037]
[-0.0531044]
[-0.01423684]


```

[-0.01723007]
[-0.05258369]
[-0.06281947]
[-0.04444267]
[-0.04631836]
[-0.06387406]
[-0.02546878]
[-0.02039166]
[-0.01681944]
[-0.01656714]
[-0.03885712]
[-0.02325963]
[-0.03097865]
[-0.02467171]
[-0.04386916]
[-0.00273399]
[-0.0426145 ]
[-0.06717044]
[-0.03531436]
[-0.04259449]
[-0.00966789]
[ 0.00194171]
[-0.04440542]
[-0.08355957]
[-0.05861557]
[-0.03367883]
[-0.09906425]
[-0.02835381]]

```

Model: "dcgan_discriminator"

Layer (type)	Output Shape	Param #
input_510 (InputLayer)	[(None, 256, 256, 3)]	0
sequential_2942 (Sequential)	(None, 128, 128, 256)	19200
sequential_2943 (Sequential)	(None, 64, 64, 512)	3278848
sequential_2944 (Sequential)	(None, 32, 32, 1024)	13111296
sequential_2945 (Sequential)	(None, 16, 16, 2048)	52436992
sequential_2946 (Sequential)	(None, 8, 8, 4096)	209731584
conv2d_1231 (Conv2D)	(None, 2, 2, 1)	102401
flatten_3 (Flatten)	(None, 4)	0
dense_509 (Dense)	(None, 1)	5

```

=====
Total params: 278,680,326
Trainable params: 278,664,966
Non-trainable params: 15,360
=====

```

```

decision_monet: [[0.9772779 ]
[0.99502265]
[0.00328818]
[0.41048917]
[0.8501729 ]
[0.9933175 ]
[0.13175988]
[0.99120814]
[0.836235 ]
[0.5142282 ]
[0.45887157]
[0.92776054]
[0.9972753 ]
[0.94121194]
[0.2626832 ]
[0.33508903]
[0.00109795]
[0.28783584]
[0.6167825 ]
[0.996071931]

```

```
[0.8977642 ]
[0.9785123 ]
[0.12174222]
[0.51355714]
[0.08464709]
[0.39661938]
[0.47304472]
[0.99269235]
[0.2628336 ]
[0.65454996]
[0.980558 ]
[0.8842257 ]]
```

Model: "dcgan_discriminator_2"

Layer (type)	Output Shape	Param #
=====		
input_511 (InputLayer)	[(None, 256, 256, 3)]	0
sequential_2947 (Sequential)	(None, 128, 128, 256)	19200
sequential_2948 (Sequential)	(None, 64, 64, 512)	3278848
sequential_2949 (Sequential)	(None, 32, 32, 1024)	13111296
conv2d_1235 (Conv2D)	(None, 14, 14, 1)	25601
flatten_4 (Flatten)	(None, 196)	0
dense_510 (Dense)	(None, 1)	197
=====		
Total params: 16,435,142		
Trainable params: 16,432,070		
Non-trainable params: 3,072		

decision_monet: [[0.6559592]

```
[0.65656734]
[0.5095648 ]
[0.5319128 ]
[0.6772772 ]
[0.62012994]
[0.5462564 ]
[0.5405856 ]
[0.52000105]
[0.5073841 ]
[0.49416474]
[0.5423713 ]
[0.5260005 ]
[0.4196489 ]
[0.55875725]
[0.5055182 ]
[0.76426464]
[0.64119744]
[0.5807137 ]
[0.5164496 ]
[0.5419576 ]
[0.4026373 ]
[0.53093106]
[0.49733457]
[0.6280133 ]
[0.60735947]
[0.5043004 ]
[0.4828298 ]
[0.66890574]
[0.5182863 ]
[0.35519886]
[0.59061927]]]
```

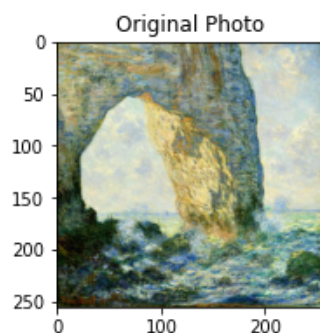
Model: "dcgan_discriminator_3"

Layer (type)	Output Shape	Param #
=====		
input_512 (InputLayer)	[(None, 256, 256, 3)]	0

sequential_2950 (Sequential)	(None, 128, 128, 128)	9600
sequential_2951 (Sequential)	(None, 64, 64, 256)	820224
sequential_2952 (Sequential)	(None, 32, 32, 512)	3278848
conv2d_1239 (Conv2D)	(None, 14, 14, 1)	12801
flatten_5 (Flatten)	(None, 196)	0
dense_511 (Dense)	(None, 1)	197
=====		
Total params: 4,121,670		
Trainable params: 4,120,134		
Non-trainable params: 1,536		

decision_monet: [[0.4549605]

[0.4429767]
 [0.53524935]
 [0.41970032]
 [0.4707817]
 [0.47729704]
 [0.44322625]
 [0.48949307]
 [0.48469207]
 [0.5047096]
 [0.47873834]
 [0.4905368]
 [0.48748466]
 [0.4487785]
 [0.46246976]
 [0.473558]
 [0.45660913]
 [0.43732178]
 [0.48177767]
 [0.46185097]
 [0.4648377]
 [0.37344056]
 [0.49059045]
 [0.49783963]
 [0.4521407]
 [0.47122034]
 [0.49529403]
 [0.5026616]
 [0.44979087]
 [0.4588775]
 [0.48230314]
 [0.49262118]]



Choice of Generator and discriminator

```
In [49]: # i_gen_disc

i_gen_disc = 4

if i_gen_disc ==0:
    with strategy.scope():
        test_generator = basic_generator()
        test_discriminator = basic_discriminator()
    ...
```

```

elif i_gen_disc ==1:
    with strategy.scope():
        test_generator = pix_generator()
        test_discriminator = pix_discriminator()
elif i_gen_disc ==2:
    with strategy.scope():
        test_generator = dcgan_generator()
        test_discriminator = dcgan_discriminator()
elif i_gen_disc ==3:
    with strategy.scope():
        test_generator = dcgan_generator_2()
        test_discriminator = dcgan_discriminator_2()
elif i_gen_disc ==4:
    with strategy.scope():
        test_generator = dcgan_generator_3()
        test_discriminator = dcgan_discriminator_3()

```

In [50]:

```

"""
# Exemple of generated images from a photo input passed through ou ba
noise_input = tf.random.normal([1, 100])

start = time.time()
to_monet = test_generator(noise_input)
print ('Time for one generated_image from photo is {} sec'.format(tim

plt.subplot(1, 2, 1)
plt.title("Original Photo")
plt.imshow(example_photo[0] * 0.5 + 0.5)

plt.subplot(1, 2, 2)
plt.title("Monet-esque Photo")
plt.imshow(to_monet[0] * 0.5 + 0.5)
plt.show()"""

```

Out[50]:

```

'\n# Exemple of generated images from a photo input passed through ou
basic generator\nnoise_input = tf.random.normal([1, 100])\n\nstart =
time.time()\nto_monet = test_generator(noise_input)\nprint (\'Time fo
r one generated_image from photo is {} sec\'.format(time.time()-star
t))\n\nplt.subplot(1, 2, 1)\nplt.title("Original Photo")\nplt.imshow
(example_photo[0] * 0.5 + 0.5)\n\nplt.subplot(1, 2, 2)\nplt.title("Mo
net-esque Photo")\nplt.imshow(to_monet[0] * 0.5 + 0.5)\nplt.show()'

```

In [51]:

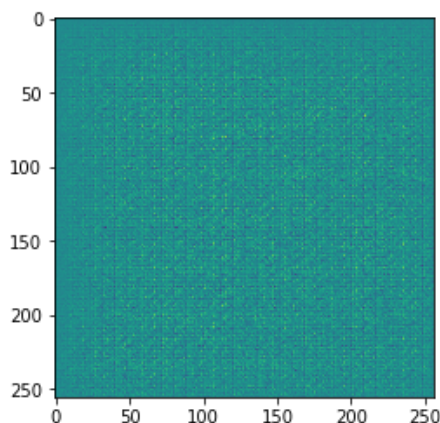
```

# Image generated from a random image
noise = tf.random.normal([1, 100])
generated_image = test_generator(noise, training=False)

plt.imshow(generated_image[0, :, :, 0])

```

Out[51]: <matplotlib.image.AxesImage at 0x7f44183b7e50>



In [52]:

```

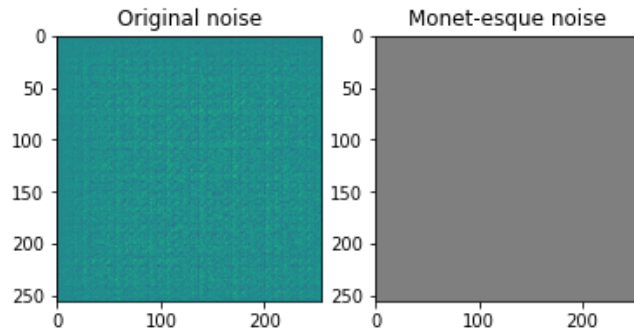
# Exemple of generated images from a photo input passed through ou ba
start = time.time()
noise_to_monet = test_generator(noise)
print ('Time for one generated_image from noise is {} sec'.format(tim

```

```
plt.subplot(1, 2, 1)
plt.title("Original noise")
plt.imshow(generated_image[0, :, :, 0])

plt.subplot(1, 2, 2)
plt.title("Monet-esque noise")
plt.imshow(noise_to_monet[0] * 0.5 + 0.5)
plt.show()
```

Time for one generated_image from noise is 0.06262421607971191 sec



In [53]: `generated_image.shape`

Out[53]: `TensorShape([1, 256, 256, 3])`

In [54]: `generated_image[0].shape`

Out[54]: `TensorShape([256, 256, 3])`

In [55]: `decision = test_discriminator(example_photo)`
`print (decision)`

```
tfn.Tensor(
[[0.50720507]
 [0.4822151 ]
 [0.5165646 ]
 [0.52958494]
 [0.46006823]
 [0.49240166]
 [0.48583955]
 [0.50692993]
 [0.48109847]
 [0.521647 ]
 [0.4479872 ]
 [0.5150014 ]
 [0.427359 ]
 [0.47798064]
 [0.4736515 ]
 [0.47881356]
 [0.4834549 ]
 [0.51654756]
 [0.4724139 ]
 [0.54947644]
 [0.4866784 ]
 [0.52292264]
 [0.5285021 ]
 [0.43360853]
 [0.44945458]
 [0.5193232 ]
 [0.51532567]
 [0.4436446 ]
 [0.47199112]
 [0.5319128 ]
 [0.43653962]
 [0.47106752]], shape=(32, 1), dtype=float32)
```

In [56]: `decision = test_discriminator(generated_image)`

```
print (decision)

tf.Tensor([[0.4998963]], shape=(1, 1), dtype=float32)
```

In []:

4.3. Buid the GAN

4.3.1. Define the loss and optimizers

In this part, we'll focus more on the architecture of the GAN, and wich loss functions will be used.

An detailed explanation of how the loss function was chosen can be found here: <https://arxiv.org/pdf/1406.2661.pdf>

Let's build the loss functions:

In [57]:

```
with strategy.scope():
    # This method returns a helper function to compute cross entropy
    # cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)
    cross_entropy = tf.keras.losses.BinaryCrossentropy(reduction=tf.k

    # from_logits=True if last layer is softmax (model output : -inf;
    # BinaryCrossentropy: 1 - P with P:[0,1]

    # we need to use reduction=tf.keras.losses.Reduction.NONE because

    """
    ValueError: Please use `tf.keras.losses.Reduction.SUM` or `tf.ker
    """
    with strategy.scope():
        loss_obj = tf.keras.losses.CategoricalCrossentropy(reduction=
        ....
        loss = tf.reduce_sum(loss_obj(labels, predictions)) * (1. / g
        ...
        Please see https://www.tensorflow.org/tutorials/distribute/custom
        """

    # UserWarning: "`binary_crossentropy` received `from_logits=True`, bu
    # "'`binary_crossentropy` received `from_logits=True`, but the `outp
    # --> If the discriminator end with sigmoid or softmax, the output wi
```

In [58]:

```
with strategy.scope():
    def discriminator_loss(real_output, fake_output):
        real_loss = cross_entropy(tf.ones_like(real_output), real_out
        fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_out
        total_loss = real_loss + fake_loss
        return total_loss
```

In [59]:

```
with strategy.scope():
    def generator_loss(fake_output):
        return cross_entropy(tf.ones_like(fake_output), fake_output)
```

In [60]:

```
with strategy.scope():
    generator_optimizer = tf.keras.optimizers.Adam(1e-4)
    discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)
```

We can test these functions:

In [61]:

```
generator_loss(noise_to_monet)
```

Out[61]: <tf.Tensor: shape=(1, 256, 256), dtype=float32, numba=


```

[15.425001 , 15.425001 , 15.425000 , ..., 15.425001 ,
 15.425014 , 15.425002 ]],

[[ 1.0323985 , 0.8507294 , 0.9270805 , ..., 0.4114161 ,
   0.41940606, 0.4194068 ],
 [ 1.0105817 , 0.8881972 , 1.0538334 , ..., 0.45989376,
   0.4936856 , 0.5066187 ],
 [ 0.9270882 , 0.8507545 , 0.96005315, ..., 0.4725187 ,
   0.5066549 , 0.5197548 ],
 ...,
 [15.424949 , 15.424966 , 15.424968 , ..., 15.425206 ,
 15.425019 , 15.42517 ],
 [15.42495 , 15.424978 , 15.42507 , ..., 15.425247 ,
 15.425162 , 15.425082 ],
 [15.424957 , 15.424999 , 15.425008 , ..., 15.42504 ,
 15.425014 , 15.425002 ]],

[[15.424949 , 15.424964 , 15.424954 , ..., 15.425 ,
 15.424959 , 15.42496 ],
 [15.424958 , 15.424981 , 15.424952 , ..., 15.424983 ,
 15.42496 , 15.424958 ],
 [15.424962 , 15.424989 , 15.424974 , ..., 15.425108 ,
 15.424994 , 15.424991 ],
 ...,
 [15.424949 , 15.424966 , 15.424968 , ..., 15.425206 ,
 15.425019 , 15.42517 ],
 [15.42495 , 15.424978 , 2.642378 , ..., 15.425247 ,
 15.425162 , 15.425082 ],
 [15.424957 , 15.424999 , 7.650754 , ..., 15.42504 ,
 15.425014 , 15.425002 ]]], dtype=float32)>

```

In [63]: `(generator_loss(noise_to_monet) + discriminator_loss(example_photo, no`

```

Out[63]: <tf.Tensor: shape=(32, 256, 256), dtype=float32, numpy=
array([[[15.257135 , 13.296581 , 14.67313 , ..., 13.468235 ,
 14.582901 , 13.449318 ],
 [14.022178 , 12.883303 , 14.770388 , ..., 14.38174 ,
 14.567663 , 14.590513 ],
 [14.536564 , 13.548608 , 13.687868 , ..., 13.064148 ,
 13.678705 , 13.632587 ],
 ...,
 [15.424949 , 14.497692 , 14.483024 , ..., 13.070788 ,
 14.263586 , 12.999909 ],
 [14.965202 , 14.408692 , 12.411208 , ..., 12.856945 ,
 11.961252 , 13.157557 ],
 [14.026953 , 12.693207 , 14.294843 , ..., 13.289939 ,
 14.27928 , 12.639741 ]],

 [[15.257135 , 13.296581 , 14.67313 , ..., 13.468235 ,
 14.582901 , 13.449318 ],
 [14.022178 , 12.883303 , 14.770388 , ..., 14.38174 ,
 14.567663 , 14.590513 ],
 [14.536564 , 13.548608 , 13.687868 , ..., 13.064148 ,
 13.678705 , 13.632587 ],
 ...,
 [15.424949 , 14.497692 , 14.483024 , ..., 13.070788 ,
 14.263586 , 12.999909 ],
 [14.965202 , 14.408692 , 12.411208 , ..., 12.856945 ,
 11.961252 , 13.157557 ],
 [14.026953 , 12.693207 , 14.294843 , ..., 13.289939 ,
 14.27928 , 12.639741 ]],

 [[15.257135 , 13.296581 , 14.67313 , ..., 13.468235 ,
 14.582901 , 8.588274 ],
 [14.022178 , 12.883303 , 14.770388 , ..., 14.38174 ,
 14.567663 , 9.723532 ],
 [14.536564 , 13.548608 , 13.687868 , ..., 13.064148 ,
 13.678705 , 13.632587 ],
 ...,
 [15.424949 , 14.497692 , 14.483024 , ..., 13.070788 ,
 14.263586 , 12.999909 ],
 [14.965202 , 14.408692 , 12.411208 , ..., 12.856945 ,
 11.961252 , 13.157557 ],
 [14.026953 , 12.693207 , 14.294843 , ..., 13.289939 ,
 14.27928 , 12.639741 ]],

```



```

[14.022178 , 12.883303 , 8.341501 , ..., 14.38174 ,
 14.567663 , 14.590513 ],
[14.536564 , 13.548608 , 13.687868 , ..., 13.064148 ,
 13.678705 , 13.632587 ],
...,
[[15.257135 , 13.296581 , 7.532205 , ..., 13.468235 ,
 14.582901 , 13.449318 ],
[14.022178 , 12.883303 , 8.341501 , ..., 14.38174 ,
 14.567663 , 14.590513 ],
[14.536564 , 13.548608 , 13.687868 , ..., 13.064148 ,
 13.678705 , 13.632587 ],
...,
[15.424949 , 14.497692 , 14.483024 , ..., 13.070788 ,
 14.263586 , 12.999909 ],
[14.965202 , 14.408692 , 12.411208 , ..., 12.856945 ,
 11.961252 , 13.157557 ],
[14.026953 , 12.693207 , 14.294843 , ..., 13.289939 ,
 14.27928 , 12.639741 ]],

[[ 8.060861 , 6.0094633, 7.4241934, ..., 5.9614434,
 7.080124 , 5.9465413],
[ 6.8149896, 5.614911 , 7.584828 , ..., 6.899195 ,
 7.1020255, 7.131344 ],
[ 7.287627 , 6.261491 , 6.455408 , ..., 5.587854 ,
 6.2195354, 6.1799693],
...,
[15.424949 , 14.497692 , 14.483024 , ..., 13.070788 ,
 14.263586 , 12.999909 ],
[14.965202 , 14.408692 , 12.411208 , ..., 12.856945 ,
 11.961252 , 13.157557 ],
[14.026953 , 12.693207 , 14.294843 , ..., 13.289939 ,
 14.27928 , 12.639741 ]],

[[15.257135 , 13.296581 , 14.67313 , ..., 13.468235 ,
 14.582901 , 13.449318 ],
[14.022178 , 12.883303 , 14.770388 , ..., 14.38174 ,
 14.567663 , 14.590513 ],
[14.536564 , 13.548608 , 13.687868 , ..., 13.064148 ,
 13.678705 , 13.632587 ],
...,
[15.424949 , 14.497692 , 14.483024 , ..., 13.070788 ,
 14.263586 , 12.999909 ],
[14.965202 , 14.408692 , 6.019862 , ..., 12.856945 ,
 11.961252 , 13.157557 ],
[14.026953 , 12.693207 , 10.407716 , ..., 13.289939 ,
 14.27928 , 12.639741 ]], dtype=float32)>

```

4.3.2. Save checkpoints

In [64]: *# Some tools to save the model during training if an exception occurs*

```

checkpoint_dir = './training_checkpoints'
checkpoint_prefix = os.path.join(checkpoint_dir, "ckpt")
checkpoint = tf.train.Checkpoint(generator_optimizer=generator_optimizer,
                                discriminator_optimizer=discriminator_optimizer,
                                generator=test_generator,
                                discriminator=test_discriminator)

```

In [65]: *# Saving model directories*

```

model_dir = './training_model'
gen_file = os.path.join(model_dir, "model_generator")
disc_file = os.path.join(model_dir, "model_discriminator")

```

In [66]: *# Tools to save images during training and we'll be used for generating*

```

def generate_and_save_images(model, epoch, test_input):
    # Notice `training` is set to False.
    # This is so all layers run in inference mode (batchnorm).
    predictions = model(test_input, training=False)

```

```

fig = plt.figure(figsize=(10, 10))

for i in range(predictions.shape[0]):
    row = int(np.around(num_examples_to_generate/5))
    col = 5

    plt.subplot(row, col, i+1)
    plt.imshow(predictions[i, :, :, 0] * 127.5 + 127.5)
    plt.axis('off')

plt.savefig('img_epochs/image_at_epoch_{:04d}.png'.format(epoch))
plt.show()

```

4.3.3. Define training loop

Here we will define the different training steps and loop to train our model. For better understanding, and because we think of using TPU, different ways to train our models will be implemented and tested.

In [67]:

```

# Some inputs used for training loop
noise_dim = 100
num_examples_to_generate = 5
BATCH_SIZE = batch_size # Comme définit en entrée
# You will reuse this seed overtime (so it's easier)
# to visualize progress in the animated GIF
seed = tf.random.normal([num_examples_to_generate, noise_dim ])

```

In [68]:

```

class GAN_class(tf.keras.Model):
    def __init__(self, generator, discriminator):
        super(GAN_class, self).__init__()
        self.generator = generator
        self.discriminator = discriminator

    # Notice the use of `tf.function`
    # This annotation causes the function to be "compiled".
    @tf.function
    def train_step_to_distribute(self, images):
        noise = tf.random.normal([BATCH_SIZE, noise_dim])

        with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
            generated_images = self.generator(noise, training=True)

            real_output = self.discriminator(images, training=True)
            fake_output = self.discriminator(generated_images, training=True)

            gen_loss = generator_loss(fake_output)
            disc_loss = discriminator_loss(real_output, fake_output)

            gradients_of_generator = gen_tape.gradient(gen_loss, self.generator.trainable_variables)
            gradients_of_discriminator = disc_tape.gradient(disc_loss, self.discriminator.trainable_variables)

            generator_optimizer.apply_gradients(zip(gradients_of_generator, self.generator.trainable_variables))
            discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator, self.discriminator.trainable_variables))

        # add codes to save losses
        #return [gen_loss, disc_loss]
        return (gen_loss + disc_loss)/2

    @tf.function
    def distributed_train_step(self, images):
        #images, generator, discriminator = dataset_inputs
        per_replica_losses = strategy.run(self.train_step_to_distribute, data_loader_args=(images,))
        return strategy.reduce(tf.distribute.ReduceOp.MEAN, per_replica_losses, axis=None)

    def train(self, dataset, epochs):
        gen_loss_list = []

```

```

gen_loss_list = []
disc_loss_list = []
train_loss_list = []
decision_list = []

for epoch in range(epochs):
    start = time.time()

    total_loss = 0.0
    num_batches = 0
    for image_batch in dataset:
        loss = self.distributed_train_step(image_batch)
        total_loss += tf.reduce_mean(loss)
        num_batches += 1
    train_loss = total_loss / num_batches

    train_loss_list.append(train_loss)

    #[gen_loss, disc_loss] = train_step(image_batch, generator, discriminator)
    #gen_loss_list.append(gen_loss)
    #disc_loss_list.append(disc_loss)

    # Produce images for the GIF as you go
    display.clear_output(wait=True)
    generate_and_save_images(self.generator,
                             epoch + 1,
                             seed)

    # Save the model every 15 epochs
    #if (epoch + 1) % 15 == 0:
    #    checkpoint.save(file_prefix = checkpoint_prefix)

    print ('Time for epoch {} is {} sec'.format(epoch + 1, time.time() - start))

    # Evaluate the model
    #loss, acc = generator.evaluate(test_images, test_labels,
    #                               num_batches=len(test_images))
    #print("Untrained model, accuracy: {:.2f}%".format(100 * acc))

    # Generate after the final epoch
    display.clear_output(wait=True)
    generate_and_save_images(self.generator,
                             epochs,
                             seed)

    # Losses saved in Lists
    #return gen_loss_list, disc_loss_list
    return train_loss_list

```

In [69]:

```

class GAN_class_bis(tf.keras.Model):
    def __init__(self, generator, discriminator):
        super(GAN_class_bis, self).__init__()
        self.generator = generator
        self.discriminator = discriminator

    def compile(self, generator_optimizer, discriminator_optimizer, generator_loss, discriminator_loss):
        super(GAN_class_bis, self).compile()
        self.generator_optimizer = generator_optimizer
        self.discriminator_optimizer = discriminator_optimizer
        self.generator_loss = generator_loss
        self.discriminator_loss = discriminator_loss

    def train_step(self, images):
        noise = tf.random.normal([BATCH_SIZE, noise_dim])

        with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
            generated_images = self.generator(noise, training=True)

            real_output = self.discriminator(images, training=True)
            fake_output = self.discriminator(generated_images, training=True)

```

```

        gen_loss = self.generator_loss(fake_output)
        disc_loss = self.discriminator_loss(real_output, fake_out

gradients_of_generator = gen_tape.gradient(gen_loss, self.gen
gradients_of_discriminator = disc_tape.gradient(disc_loss, se

self.generator_optimizer.apply_gradients(zip(gradients_of_gen
self.discriminator_optimizer.apply_gradients(zip(gradients_of

# add codes to save losses
#return [gen_loss, disc_loss]
return {
    "gen_loss": gen_loss,
    "disc_loss": disc_loss,
    "decision": fake_output
}

```

In [70]:

```

class GAN_class_2(tf.keras.Model):
    def __init__(self, generator, discriminator):
        super(GAN_class_2, self).__init__()
        self.generator = generator
        self.discriminator = discriminator

    def compile(self, generator_optimizer, discriminator_optimizer, g
        super(GAN_class_2, self).compile()
        self.generator_optimizer = generator_optimizer
        self.discriminator_optimizer = discriminator_optimizer
        self.generator_loss = generator_loss
        self.discriminator_loss = discriminator_loss

    @tf.function
    def train_step(self, inputs):
        def step_fn(self, images):

            noise = tf.random.normal([BATCH_SIZE, noise_dim])

            with tf.GradientTape() as gen_tape, tf.GradientTape() as
                generated_images = self.generator(noise, training=True

            real_output = self.discriminator(images, training=True
            fake_output = self.discriminator(generated_images, tr

            gen_loss = self.generator_loss(fake_output)
            disc_loss = self.discriminator_loss(real_output, fake

            gradients_of_generator = gen_tape.gradient(gen_loss, self
            gradients_of_discriminator = disc_tape.gradient(disc_loss

            self.generator_optimizer.apply_gradients(zip(gradients_of
            self.discriminator_optimizer.apply_gradients(zip(gradient

            # add codes to save losses
            #return [gen_loss, disc_loss]
            return {
                "gen_loss": gen_loss,
                "disc_loss": disc_loss
            }

        per_replica_losses = strategy.run(step_fn, args=(inputs,))
        return strategy.reduce(tf.distribute.ReduceOp.MEAN, per_repli
                                axis=None)

    """
    @tf.function
    def distributed_train_step(self, images):
        #images,generator,discriminator = dataset_inputs
        per_replica_losses = strategy.run(self.train_step_to_distribu
        return strategy.reduce(tf.distribute.ReduceOp.MEAN, per_repli
                                axis=None)
    """

```

5. Train the GAN

```
In [71]: EPOCHS = 20 #1200

make_submission = 0 # 0: no generation of images for submission, 1: g

# Choice of GAN strategy

iGAN = 1

# 0: monet_gan from GAN_class using distribute and train
# 1: monet_gan_bis from GAN_class_bis using compile and fit
# 2: monet_gan_2 from GAN_class_2 using compile and fit and distribut
```

```
In [72]: #with strategy.scope():
# [gen_loss_list, disc_loss_list] = train(monet_ds, EPOCHS, test_genera
# train_loss = train(monet_ds, EPOCHS, test_generator, test_discriminato
```

```
In [73]: #len(gen_loss_list)
```

```
In [74]: #len(disc_loss_list)
```

```
In [75]: """
fig = plt.figure(figsize =(10, 7))
plt.plot(gen_loss_list)
plt.plot(disc_loss_list)

plt.plot(np.mean(gen_loss_list),marker='o')
plt.plot(np.mean(disc_loss_list),marker='o')

plt.plot([0,100],[np.mean(gen_loss_list),np.mean(gen_loss_list)])
plt.plot([0,100],[np.mean(disc_loss_list),np.mean(disc_loss_list)])

plt.legend(['gen_loss_list','disc_loss_list',
            'mean_gen_loss_list','mean_disc_loss_list'])

plt.xlabel("Image number")
plt.ylabel("Generator and Discriminator losses")

plt.show()
"""
```

```
Out[75]: \nfig = plt.figure(figsize =(10, 7))\nplt.plot(gen_loss_list)\nplt.p  
lot(disc_loss_list)\n\nplt.plot(np.mean(gen_loss_list),marker='\o')  
\nplt.plot(np.mean(disc_loss_list),marker='\o')\n\nplt.plot([0,100],  
[np.mean(gen_loss_list),np.mean(gen_loss_list)])\nplt.plot([0,100],[n  
p.mean(disc_loss_list),np.mean(disc_loss_list)])\n\nplt.legend(['gen  
_loss_list','disc_loss_list',\n\n        \n        'mean_gen_loss_list  
\n','mean_disc_loss_list'])\n\nplt.xlabel("Image number")\nplt.ylabel  
l("Generator and Discriminator losses")\n\nplt.show()\n'
```

Restore latest checkpoint

```
In [76]: if iGAN ==0:
           import os
           os.mkdir("img_epochs")
```

```
In [77]: checkpoint.restore(tf.train.latest_checkpoint(checkpoint_dir))
```

```
Out[77]: <tensorflow.python.training.tracking.util.InitializationOnlyStatus at 0x7f4444040450>
```

```
In [78]: if iGAN ==0:
         with strategy.scope():
             monet_gan = GAN_class(test_generator, test_discriminator)
```

```
In [79]: %%debug
         if iGAN ==0:
             #!/ mkdir img_epochs
             train_loss_result = monet_gan.train(monet_ds, EPOCHS)
```

```
In [80]: if iGAN ==0:
         fig = plt.figure(figsize =(10, 7))
         plt.plot(train_loss_result)

         plt.plot(np.mean(train_loss_result),marker='o')

         plt.plot([0,len(train_loss_result)],[np.mean(train_loss_result),n

         plt.legend(['train_loss_result','mean_train_loss_result'])

         plt.xlabel("Image number")
         plt.ylabel("Generator and Discriminator losses")

         plt.show()
```

In []:

```
In [81]: if iGAN ==1:
         with strategy.scope():
             monet_gan_bis = GAN_class_bis(test_generator, test_discrimina
             monet_gan_bis.compile(generator_optimizer, discriminator_opti
```

```
In [82]: #!/ monet_gan_bis.generator.trainable_variables
```

```
In [83]: if iGAN ==1:
         history = monet_gan_bis.fit(monet_ds, epochs = EPOCHS)
```

```
Epoch 1/20
9/9 [=====] - 358s 39s/step - gen_loss: 3.30
22 - disc_loss: 1.1813 - decision: 0.2271
Epoch 2/20
9/9 [=====] - 337s 37s/step - gen_loss: 3.90
19 - disc_loss: 0.6762 - decision: 0.2030
Epoch 3/20
9/9 [=====] - 336s 37s/step - gen_loss: 2.25
56 - disc_loss: 1.1819 - decision: 0.3464
Epoch 4/20
9/9 [=====] - 338s 38s/step - gen_loss: 2.85
02 - disc_loss: 1.2024 - decision: 0.3168
Epoch 5/20
9/9 [=====] - 342s 38s/step - gen_loss: 2.17
72 - disc_loss: 2.7532 - decision: 0.4727
Epoch 6/20
9/9 [=====] - 339s 38s/step - gen_loss: 3.18
85 - disc_loss: 3.1129 - decision: 0.4184
Epoch 7/20
9/9 [=====] - 367s 41s/step - gen_loss: 2.93
64 - disc_loss: 3.9495 - decision: 0.4843
Epoch 8/20
9/9 [=====] - 346s 38s/step - gen_loss: 3.74
47 - disc_loss: 2.5124 - decision: 0.3858
Epoch 9/20
9/9 [=====] - 340s 38s/step - gen_loss: 2.81
18 - disc_loss: 2.8058 - decision: 0.4432
Epoch 10/20
```

```

9/9 [=====] - 341s 38s/step - gen_loss: 2.86
02 - disc_loss: 2.8712 - decision: 0.4726
Epoch 11/20
9/9 [=====] - 337s 37s/step - gen_loss: 2.93
73 - disc_loss: 2.5576 - decision: 0.4262
Epoch 12/20
9/9 [=====] - 339s 38s/step - gen_loss: 2.35
27 - disc_loss: 2.1394 - decision: 0.4017
Epoch 13/20
9/9 [=====] - 342s 38s/step - gen_loss: 2.49
50 - disc_loss: 1.6576 - decision: 0.3979
Epoch 14/20
9/9 [=====] - 346s 39s/step - gen_loss: 2.89
73 - disc_loss: 1.2002 - decision: 0.3150
Epoch 15/20
9/9 [=====] - 346s 38s/step - gen_loss: 2.55
95 - disc_loss: 1.2462 - decision: 0.3341
Epoch 16/20
9/9 [=====] - 342s 38s/step - gen_loss: 2.41
17 - disc_loss: 1.5755 - decision: 0.3549
Epoch 17/20
9/9 [=====] - 345s 38s/step - gen_loss: 2.32
42 - disc_loss: 1.5840 - decision: 0.3456
Epoch 18/20
9/9 [=====] - 369s 41s/step - gen_loss: 2.32
61 - disc_loss: 1.5471 - decision: 0.3317
Epoch 19/20
9/9 [=====] - 379s 42s/step - gen_loss: 2.40
78 - disc_loss: 1.2264 - decision: 0.2761
Epoch 20/20
9/9 [=====] - 376s 42s/step - gen_loss: 2.56
91 - disc_loss: 0.9854 - decision: 0.2123

```

In [84]: `#monet_gan_bis.generator(noise).shape[0]`

```

In [85]: if iGAN ==1:
fig = plt.figure(figsize =(10, 7))
#plt.plot([0,len(history.history['gen_loss'])], history.history['
plt.plot(np.mean(history.history['gen_loss'], axis=1))
plt.plot(np.mean(history.history['disc_loss'], axis=1))
plt.plot(np.mean(history.history['decision'], axis=1))

plt.plot(np.mean(np.mean(history.history['gen_loss'], axis=1)),ma
plt.plot(np.mean(np.mean(history.history['disc_loss'], axis=1)),m
plt.plot(np.mean(np.mean(history.history['decision'], axis=1)),ma

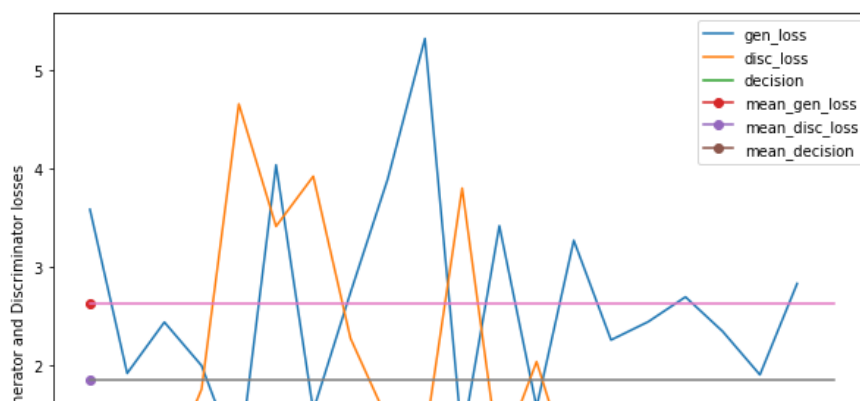
plt.plot([0,len(np.mean(history.history['gen_loss'], axis=1))],[n
plt.plot([0,len(np.mean(history.history['disc_loss'], axis=1))],[
plt.plot([0,len(np.mean(history.history['decision'], axis=1))],[n

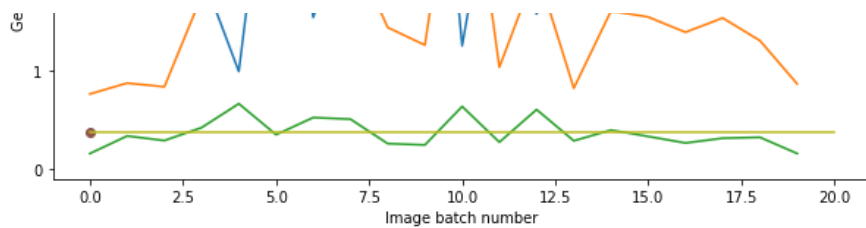
plt.legend(['gen_loss','disc_loss','decision','mean_gen_loss','me

plt.xlabel("Image batch number")
plt.ylabel("Generator and Discriminator losses")

plt.show()

```





```
In [86]: #history.history['gen_loss']
```

```
In [87]: #np.mean(history.history['gen_loss'], axis=1)
```

```
In [88]: #history.history['gen_loss'][0]
```

```
In [89]: #history.history['gen_loss'][0][-1]
```

```
In [90]: #history.history['decision']
```

```
In [91]: #np.mean(history.history['decision'], axis=1)
```

```
In [92]: #history.history['gen_loss'][-1]
```

```
In [93]: if iGAN ==2:
    with strategy.scope():
        monet_gan_2 = GAN_class_2(test_generator, test_discriminator)
        monet_gan_2.compile(generator_optimizer, discriminator_optimi
```

```
In [94]: if iGAN ==2:
    monet_gan_2.fit(monet_ds, epochs = EPOCHS)
```

```
In [ ]:
```

```
In [95]: # #Stop runing
# import sys
# sys.exit()
```

6. Test the GAN

```
In [96]: if iGAN == 0:
    generator_to_test = monet_gan.generator
elif iGAN == 1:
    generator_to_test = monet_gan_bis.generator
elif iGAN == 2:
    generator_to_test = monet_gan_2.generator

fig = plt.figure(figsize=(10, 10))

#num_examples_to_generate = 10
for i in range(generator_to_test(seed).shape[0]):
    raw = int(np.around(num_examples_to_generate/5))
    col = 5

    plt.subplot(raw, col, i+1)
    plt.imshow(generator_to_test(seed)[i, :, :, 0] * 127.5 + 127.5)
    plt.axis('off')
```



```
PIL.Image.open('img_epochs/image_at_epoch_{:04d}.png')
```



Création de GIF

```
In [97]: if iGAN ==0:
# Display a single image using the epoch number
def display_image(epoch_no):
    return PIL.Image.open('img_epochs/image_at_epoch_{:04d}.png').
```

```
In [98]: if iGAN ==0:
display_image(EPOCHS)
```

Animated Gif

```
In [99]: if iGAN ==0:
import imageio.v2 as imageio

anim_file = 'dcgan.gif'

with imageio.get_writer(anim_file, mode='I') as writer:
    filenames = glob.glob('img_epochs/image*.png')
    filenames = sorted(filenames)
    for filename in filenames:
        image = imageio.imread(filename)
        writer.append_data(image)
    image = imageio.imread(filename)
    writer.append_data(image)
```

```
In [100... if iGAN ==0:
# To generate GIFs
! pip install imageio
! pip install git+https://github.com/tensorflow/docs
```

```
In [101... if iGAN ==0:
import tensorflow_docs.vis.embed as embed
embed.embed_file(anim_file)
```

```
In [ ]:
```

7. Save model

```
In [102... # Saving model

model_dir = './training_model'
gen_file = os.path.join(model_dir, "model_generator")
disc_file = os.path.join(model_dir, "model_discriminator")
```

```
In [103... # Methods for saving the generated folder of saved trained model in a
from zipfile import ZipFile
import os

def save_model_zip(test_folder, zip_file):
```

```
#zip_file = "test_folder.zip" # zip file name
directory = test_folder

with ZipFile(zip_file, 'w') as zip_obj:
    for path, directories, files in os.walk(directory):
        for file in files:
            file_name = os.path.join(path, file)

            zip_obj.write(file_name) # zipping the file

print("Contents of the zip file:")
with ZipFile(zip_file, 'r') as zip_obj:
    zip_obj.printdir()
```

In [104...

```
# Saving actions if using CPU/GPU
"""
monet_gan.generator.save(gen_file) # + str(epoch))
monet_gan.discriminator.save(disc_file) # + str(epoch))

zip_file = os.path.join(model_dir, "gan_model_file.zip")
#save_model_zip(gen_file, zip_file)
```